CREATING A UNIVERSAL INFORMATION LANGUAGE

OSCAR BENDER-STONE

ABSTRACT. Welkin is a formalized information language. We introduce its use cases and rigorously define its syntax and semantics. From there, we introduce the bootstrap, making Welkin completely self-contained. [TODO: determine how to phrase soundness and incompleteness. Should we include these?].

CONTENTS

1. Introduction . . . 1
2. Rationale . . . 5
   2.1. General Requirements . . . 5
   2.2. Units . . . 6
   2.3. Mechanizing Information . . . 7
   2.4. Base Operations . . . 8
   2.5. Bootstrapping . . . 9
3. Foundations . . . 10
   3.1. Base Rules . . . 10
   3.2. PRA . . . 13
   3.3. Expressing PA and Feferman Reflection . . . 14
   3.4. Coherency and Information . . . 15
4. Syntax . . . 15
   4.1. Words . . . 15
   4.2. Terminals . . . 15
   4.3. EBNF Notation and Parse Trees . . . 17
   4.4. The Welkin Grammar . . . 17
   4.5. Proof of Unambiguity . . . 18
5. Semantics . . . 22
   5.1. Abstract Syntax Tree (ASTs) . . . 23
   5.2. Unified IDs . . . 24
   5.3. Unification . . . 24
   5.4. Queries and Information . . . 25
6. Information Organization . . . 26
   6.1. Universal Systems . . . 26
   6.2. Localized Size Compression . . . 27
7. Conclusion . . . 27
   7.1. Comparisons to Existing Literature . . . 27
   7.2. Programming Languages and Formal Verification . . . 28
   7.3. Mathematical and Scientific Knowledge . . . 28
   7.4. Humanities . . . 28
References . . . 29

DRAFT

1. INTRODUCTION

Information Management is the study of systematically storing and organizing data. With the massive growth of data, particularly on the web, simply storing data to *use* it. Raw databases are increasingly structured into organized *information bases* to prove

---

Date: March 3, 2026.

context surrounding the source and nature of data. The conceptual foundation for this process often traces back to Ackoff [Ack89]. His data-to-wisdom hierarchy posits that information is the process of transforming raw data into structured content. In turn, this content can be used to answer user queries. However, the term "information" has not been standardized in the literature and is used with distinct meanings. For example, Buckland [Buc91] recognizes three different ways information is interpreted: as a *process*, as *knowledge*, and as *a thing*. While each of these are important, Buckland emphasizes that to *use* information, one must work with information as a thing—a tangible record like a document or digital file. Another author, Floridi, builds upon this tangible idea by requiring data be truthful [Flo10]. Information must not only be in concrete records, but also accurately represent a topic of interest. As shown in this small sample of authors, information is a rich term, with an active pursuits to best manage it.

Despite these extensive ideas, applying Information Management in real world projects reveals ongoing problems. One of these issues is the *depth* of domains. For example, at major retailers like Amazon, inventory and recommendation systems use immense amounts of data to power personalized recommendations to users [SL17]. Another issue is the immense breadth of different subjects. As a well known example, Wikipedia struggles to constantly update its vast and interconnected web of knowledge [Mes+15]. Along with the depth and breadth of different disciplines, standardization is complex. Most systems have their own unique data formats, so making **interoperable** formats is hard, which is the ability to transfer one data format to another. For instance, transferring information between two medical platforms is notoriously difficult, even if they store the *same* information about a patient [Rei17]. These problems continue to be a challenge in effectively utilizing frameworks for information.

In consideratino of these issues, several solutions partially resolve some of them, but not completely. The following provides a small list of major existing solutions. Reading these is optional and not required to understand this thesis. We will not revisit these until the conclusion, see Section 7.1.

- **Resource Description Framework (RDF) Triple Stores:** These systems serve as the primary implementations of an internet standard called the Web Ontology Language (OWL) [Hit+09]. By storing information as triples $\text{subject} \text{ — } \text{predicate} \rightarrow \text{object}$. From there, the language can then enforce rules and ensure new relationships derived are *only* from previously given relationships and valid rules. OWL has been highly successful in providing a machine-readable format to websites across the internet [Hit21]. Despite their widespread adoption, RDF stores struggle with extensive sources of data. Enforcing logical rules can be expensive [Zhu+23]. Additionally, because OWL treats missing sources of data as "unknown" rather than "false", it often fails to resolve conflicts between two sources [Hit21].
- **Labeled Property Graphs (LPGs):** This is an architecture defined by its ability to store rich, internal metadata within its components. Building on a network, consisting of nodes and edges between them, LPGs enable both nodes (entities) and edges (relationships) to possess a set of key-value pairs known as *properties* [Ang18]. The internal metadata allows systems to efficiently traverse networks, as attributes can be filtered and weighed on a single query pass. Due to their operational efficiency, LPGs are the primary choice in

large scale commercial products, particularly real-time fraud detection tools and recommendation systems [RWE15]. However, LPGs are siloed by the lack of standards, making interoperability between databases challenging. Furthermore, LPGs lack an extensive logical engine, resulting in external tooling or complex query scripts [RWE15].

- **Cyc:** Designed to encompass broad subjects, Cyc is the largest proprietary database of knowledge, or *knowledge base*, known to date [Len95]. It achieves reliable queries by providing a logical engine, based on millions of hard coded "common sense" rules. However, despite its impressive scope, it relies on human experts to manually encode every rule. This causes a knowledge acquisition bottleneck, limiting how well the database scales [Har07]. Another issue is the absence of an active open standard, a result of the publicly available version, OpenCyc, being discontinued in 2017 [KBp].
- **fcaR (Formal Concept Analysis in R):** This tool is a modern implementation for the field of Formal Concept Analysis (FCA). FCA studies relationships through *formal concepts*. The central idea is the *concept lattice*, a hierarchical data structure that maps objects to attributes in a fixed domain. Given only a database, fcaR can automatically identify the underlying core concepts and derived concepts [MDV26]. While being highly effective in specialized domains, concept lattices can become impractically large as the number of attributes increases [HS13]. This makes it difficult to interpret results when applied to large-scale databases or when bridging disparate domains [HS13].

This thesis introduces a language to resolve these issues for Information Management. I call this language **Welkin**, based on an old German word meaning cloud [Dic25]. This word represents the expansive nature of the language.

The core result of this thesis is proving that Welkin fulfills three goals: it is **universal**, **scalable**, and **standardized**. For details, see Table 1. The fundamental building block is based on **representations**, containing a relationships between a **sign** representing a **sign** in a **context**.[1] Contexts provide flexibility to define domain terminology, and are provided rules inspired by McCarthy's work in AI systems [McC93]. Moreover, truth is defined relative to contexts and enables the user to recognize *under which conditions* a statement is true. In this way, the information base keeps conflicting claims contained, providing a clear separation from other contexts.

Building on representations, information is defined as a property preserved under equivalences relative to a context. To make the information base scalable, the base operations are kept as simple as possible, and a procedure to adaptively organize information. The simplicity of the language aids in standardization as well. For the highest reassurance, we create a restricted, finite subset of the language. This subset facilitates proofs in this thesis and enables ease of implementation.

[1]There are similarities with representations and Peirce's semiotics. This theory studies the relationship between a symbol, the object it represents, and the interpretation that provides that meaning [Atk23]. Our notion is different in that contexts are a generalized interpretant, see Section 2.2.

Goal	Name	Description
Goal 1	**Universality**	The language must enable any user created parameters, whose symbolic representation is accepted a computable function. Every computable function must be definable in the language.
Goal 2	**Scalability**	The database must appropriately scale to broad representations of information. Local queries must be efficient. Certificates must be available to prove cases where optimal representations have been achieved.
Goal 3	**Standardization**	The language needs a rigorous and formal specification. Moreover, the bootstrap must be formalized, as well as an abstract machine model. The grammar and bootstrap must be fixed to ensure complete forwards and backwards compatibility. Certificates must be reliably checked and rely on a low level of trust, or a small Trusted Computing Base.

TABLE 1. Goals for the Welkin language.

This thesis is organized according to Table 2.

DRAFT

Section Number	Title	Description
Section 2	**Rationale**	Introduces Welkin at a high level, with guiding examples.
Section 3	**Foundations**	Introduces the core theory behind Welkin, which is the starting point to specify the user-facing language.
Section 4	**Syntax**	Provides the grammar and proof that it is unambiguous.
Section 5	**Semantics**	Explains how ASTs are validated and processed. Develops representations and coherency, and connects these to a working definition of information.
Section 6	**Information Organization**	Develops a Greedy algorithm to locally optimize information. Creates a certificate that demonstrates when a representation is optimal relative to the current information database.
Section 7	**Conclusion**	Concludes with comparisons to existing theories and possible applications, particularly in programming languages and broader academic knowledge management.

TABLE 2. Organization for the thesis.

DRAFT

2. RATIONALE

In this section, we justify the design of Welkin.

2.1. General Requirements.

The main purpose of an information base is to store information and answer user queries based on available information. To provide an upper bound on our requirements, we require two important properties:

1) The information base must be mechanized, which means only "effective operations" are allowed. We make "effective operations" precise with the technical notion of *partial computable functions*. Partial computable functions are computer programs that can process a finite input, and that either stop on an input or continue forever. The bedrock for these functions is the notion of a *Turing machine*, which models every current form of computation. For more information, see [Tur36]. To obtain universality, we need to express *every* one of these machines.
2) The information base must provide a mechanism to *ground* symbols defined by users. This stems from the Symbol Grounding Problem, posed by Harnard [Har90]. To avoid storing physical objects or such, users should have a way to provide *meaning* to symbols, or a way to *represent* their topic of interest.

Combined together, we can now state universality as follows: *an information base is universal if it can represent anything representable by a partial computable function.* We will clarify how we define "representable" in the next section.

2.2. **Units.**

[TODO[MEDIUM]: double check all examples!]

The first core concept in Welkin is the notion of a **handle**. Handles are left as free parameters in the theory[2] In an information base, handles are uniquely identified through their **key**, directly inspired by database theory. Keys are with triples of IDs $(\mathrm{UID}, \mathrm{RID}, \mathrm{HID})$, whose contents are the **user ID**, **revision ID**, and **handle ID**, respectively. Revisions ensure that storing axioms is **immutable**, or that a particular handle does not change over time. This is *not* a restriction on dynamic entities, but rather an indication to the information base to determine a particular version of a handle.

Example 2.2.0. In a scientific experiment, a handle could be an observation or experimental data. The unit is then written as a symbol, say u, and is *implicitly* bound to this meaning. To distinguish from other symbols, say v, the computational content is analyzed.[TODO[MEDIUM]: expand out this example!]

Indexing handles is only one important aspect of handles. Another is that handles are precisely by their relationships. These are written as **representations** $a \mathrel{—} c \to b$, which means that a represents b in context c. Here, we call a the **sign**, b the **referent**, and c the **context**. One can interpret this characterization as follows: handles are defined by how they are and are *not* restricted. This idea is directly inspired from Fine's idea of arbitrary objects [Fin85], and allows refinements to a study of interest.

Example 2.2.1. A more looser example is a user written journal for therapy sessions, containing information about daily habits and emotions. While neither of these are stored in the information base, their handles are, via units habit and $\mathrm{emotions}$ in a context $\mathrm{journal}$. Moreover, multiple revisions of the journal can be made with dates or other unique IDs.

We represent handles in Welkin using an extended notion called a **unit**. A unit is either a handle, a finite collection of units, or a representation. Collections of units can have a uniquely generated name by the information base called a **block**[3].

Example 2.2.2.

Units define their own **context**, which means the unit provides its own names and representations. Writing $a \mathrel{—} c \to b$, therefore, only applies to the *fixed* context c.

[TODO[SMALL]: use better names for entities/businesses/etc]

[2]For mathematicians, this is analogous to how key notions are left undefined in Hilbert's formalism of geometry, or how a "set" is an undefined notion in set theory.

[3]In programming languages, this is analogous to an anonymous function.

Example 2.2.3. A business could represent their operations using a unit $\mathrm{business}$ that contains units for their workers and ledgers. This allows another unit, say $\mathrm{business2}$, to contain its *own* label $\mathrm{workers}$ that is separate from the one in $\mathrm{business}$. In addition to separate labels, these contexts can have *distinct* rules, such as those for how business operations are performed.

2.3. **Mechanizing Information.**

From the notion of a unit, we practically define information on a unit v as a unit u which *correctly identifies v from any other unit*. We formalize this as an invariant based on an equivalence relation on units, determined by the given set of representations (see Section 5). This notion corresponds to Burgin's idea of information as an *operator* that transforms a system, and is closely to Bateson's famous quote that "information is a difference that makes a difference" [Bat00]. Our practical distinction between information and knowledge is that we *use* information, but users can assert their own notions of these terms by creating restricted contexts. We connect our notion of information back through Algorithmic Information Theory by showing that the theoretical notion of conditional information content *precisely* coincides with the size of information, counted in bits (up to a constant) [TODO: link to this result. Important! And confirm this result! Currently unchecked.].

Within a fixed revision, handles, by being defined by how they are restricted, are *exactly* described by the consequences of their relationships. We describe this as a syntactic rendering of truth through the notion of a truth management system. Here, we define a truth management system to consist of a set of axioms along with inference rules, whose derivations are accepted by some Turing machine. Welkin proves that not only is *any* truth management system.

There are two primary inference rules, which we informally describe now (and postpone nested contexts until Section 5):

- **Internal Transitivity:** if u represents v in context c and v represents x in context c, then u represents x in context c.
- **Lifting:** if u represents v in context c and p represents q in context v, then within context c, p represents q in *nested* context u.

Transitivity is a common axiom in many systems. It is particularly efficient when used in contexts, as shown by several projects [Len+90], [GWW07]. Lifting is a notion closely tied to McCarthy's notion of lifting, but formulated with representations. The idea is, if a represents b in context c, then a provides an *abstraction* for b relative to c. Wherever b is involved in a relationship, a can be used *instead*, acting as a proxy relative to c. Contexts provide an import mechanism as well, thereby providing Welkin with a module based system. We provide a simplified version of one McCarthy's examples below. For more existing examples in the literature, refer to [BF95].

DRAFT

Example 2.3.4. (McCarthy's Above Theory). Consider a set of physical blocks. Denote the unit `above` to represent the relation `block A is above block B`. We want to say this could be `on` to mean `block A is on block B`, or `floating_above` to mean `block A is floating above block B`. One way to state is as two axioms: and we could state as an axiom: `A - on -> B` implies `A - above -> B` and `A - floating_above -> B` implies `A - floating_above -> B`. Lifting provides a more economical approach *without* needing to provide explicit terms, or provides a "point-free" rendering. We could that, within `above`, `above` represents `on` and `above` represents `floating_above`. In other words, `above` *precisely abstracts* from the precise relationship between the two blocks.

Another example concerns contexts in the presence of unique objects.

Example 2.3.5. Consider a biological survey of home pets, denoted by context `Survey`. Suppose there are two units in `Mammal`, say a dog `Fido` and a cat `Lucy`. We could state that `Fido` *represents* `Mammal`, to say that `Fido` acts as a stand-in for a mammal in the survey. The same could be said with `Lucy` representing `Mammal`. Note that, depending on the context, the referent need *not* be more refined than the sign; this is an intenional design choice for flexibility. With a another context `Taxonomy`, we may naturally state that `Mammal - Taxonomy -> Animal`, and we could say `Survey - instanceof -> Taxonomy`. Thus, the lifting rule implies in context `instanceof`, `Mammal - Survey -> Animal`. Using transitivity, we obtain `Fido - Survey -> Animal` and `Lucy - Survey -> Animal` in context `instanceof`. We can interpret this example as saying that the *relationships* of a general taxonomy are witnessed by a specific taxonomy, namely `Survey`, and that they propagate through contexts.[4]

The remaining rules are primarily for efficiency, enabling users to have positionally invariant information, see Definition 3.1.11.

2.4. **Base Operations.**

Now, units and information themselves could be expressed in infinitely many languages, with slightly different syntax or semantics. Welkin is carefully designed to be a *minimal* expression of these concepts, with minimal friction to express any other universal information base. These include:

- In ASCII, `a - b -> c` for representations. These can be interpreted *as* rewrite rules, depending on the context.
- Traditional braces `{ }` to denote closed definitions of contexts, inspired by the C programming language.
- Paths via dots `.` that is inspired by the Python programming language. Relative paths are denoted with multiple dots `...`, and absolute imports are prefixed with `#`. Subsets of units can be written via `u.{v, x}`, or even `u.{v --> x}` to refer to a subset of representations.
- Imports are done through `@u`, which takes all subunits of `u` and puts them into the current scope. In other words, the implementation *implicitly* adds denotations `v <--> a.v` for each subunit `v` of `u`. This is motivated by the abundance of boolean logic in classical mathematics and computer science, as any finite

[4]In programming languages, particularly C++ [Str13] and Java [AGH05], this property is known as "upcasting", with a less refined base class being replaced with a more refined subclass. The reverse is "downcasting". Both are supported by Welkin for full expressivity.

DRAFT

circuit can be expressed in terms of `and` and `not`. Selecting specific subunits can be done via `@u.{v, x}`.

- Imports can be *negated* via the notation `~@u`, and specific units can be negated through `~u`. This is an uncommon feature of most programming languages, appearing primarily in Haskell and CSS. While potentially opaque, Welkin provides robust definitions to ensure that negated forms can be easily translated into more explicit ones *and* back again. [TODO[SMALL]: provide link, probably to bootstrap or so?]
- Comments *are* strings can be treated as any unit. No comments need to be removed in the files and can *enhance* the study of new subjects.[5] These units can *then* be analyzed by others to promote translations to other human languages, or be studied through the *overarching relationships* in the units.

In general, the minimal restricted keywords is crucial for providing support for other languages. An implementation of Welkin will contain a small section of ASCII encoding for easier standardization, but the rest of the program can be done *entirely* in the user's native language. This is a novel feature in most programming languages, which are either predominantly English or are fine tuned for specific human languages. [TODO: cite source on this about most programming languages being in English, as well as programming languages written in *different* human languages. Would be useful to have.]

2.5. **Bootstrapping.**

There are two important questions for implementing Welkin:

- How do we practically implement the operations?
- How do we enable any "computably checkable proof"?

On the first issue, the author previously went to manually describing the syntax and semantics, introducing notions from parsing theory and compilers. While this format is traditional for a programming language, this postponed the main rules and made the language specification harder to follow. Moreover, the author sought to explain the minimal set of axioms required for the system. This connects to the second issue in a larger way, closely tied into formal methods.

In the formal methods literature, a **Trusted Computing Base (TCB)** is the set of axioms and code used to implement a program. Axiomatically, most existing proof assistants have a fixed set of rules, with early proponents arguments towards having a fixed TCB (see)...

To effectively create any formal system, having *some* notion of IDs is necessary in a Trusted Computing Base. Checking the trace of a Turing machine can be done with a finite certificate, *but* checking for non-halting in the general case requires a level of induction, expressed through IDs. Without IDs, we cannot even *state* essential constructs or theorems, including the Gödel encodings.

To resolve both issues, they are combined into approach analogous to dependent type theory in the sense of "proofs as programs", but in a simpler way. We call this process "bootstrapping", analogous to bootstrapping a programming language from another through successive iterations, with each iteration building upon previous

[5] Contexts can mimic regular comments, but Welkin aims to be inspectable by users. Encapsulation or private information can be enforced through *rejecting* specific contexts or only accepting certain ones.

DRAFT

ones to define new language features. We also define the notion of "computably checkable proof" through reflection, providing a partial translation from Feferman's work (partial in the sense of not requiring the *same* semantics as first order logic) [Fef62]. While a *single* formal system cannot enumerate through these proofs by Gödel incompleteness theorem, our theory provides the most flexibility and enables the *potential* of expressing any recursive ordinal, i.e., the set of axioms allowed is *not* fixed but depends on context.

Taken together, the steps for bootstrapping Welkin are:

1) Formalize the base rules on units. Using English as our meta-language, we prove that our rules can define the S and K combinators as any combination of terms, and thus is Turing complete. Having Turing completeness ensures the language is expressive enough (as per Goal 1), but we will use more direct notions in terms of the rules for ease of presentation. Another rule we add is for a specific verifier for proofs, which we can prove in the system is correct. We will later show this is *not* a restriction and applies to any computably checkable proof.
2) Provide the base meta-theory for the language *in* Welkin. This will be PRA, written in a combinator based form. We will show the main axioms of PRA hold.
3) Prove that Peano Arithmetic is provably sound in a weak sense using a method developed by Artemov [Art24]. From there, we carry out Feferman's Reflection theorem written with our notions.
4) Define handles and information. We also justify information organization to promote the *most* optimal paths to prove things, *if* proofs are provided. This is deeply connected to Blum's speedup theorem and is essential for proving Goal 2.
5) Define the rules for path resolution and global IDs. These use a similar result to syntax descriptions and are significantly easier to read. [NOTE: emphasis on WIP here! Not yet developed in thesis !]
6) Finally, define the syntax through an *invertible syntax description*. This idea, pioneered by Rendel and Ostermann [RO10], provides a combinator approach to define how concrete syntax, that is user written, is converted into abstract syntax, an abstracted form, *and back*. Along with the syntax, the encoding is given *itself* through a representation, namely a table provided in this thesis.

3. Foundations

This section discuss the foundations of Welkin, as follows:

1) We define units and their rules.
2) We define information.

3.1. **Base Rules.**

For notation, we will write $a \equiv b$ to mean that a is definitionally equivalent to b. Moreover, we will distinguish between *defining* a term as finite and *practically enforcing* it is finite. For a thorough discussion, see ?.

DRAFT

Definition 3.1.0. A **bit** is the symbols 0 or 1. A **binary word** is either the symbol ε (the **empty word**), and if w is a finite binary word, so are $w.0$ and $w.1$, where . is the symbol for **concatenation**. Nothing else is a word.

Definition 3.1.1. Equality $=$ and inequality $\neq$ on words w_1, w_2 is defined recursively:

- **Base case:** $\varepsilon = \varepsilon$.
- **Recursive step:** suppose $w_1 = b_1.w_{1'}$ and $w_2 = b_2.w_{2'}$, where b_1, b_2 as bits and $w_{1'}, w_{2'}$ are words. Then $w_1 = w_2$ if and only if $b_1 = b_2$ and $w_1' = w'2$. Moreover, $w_1 \neq w_2$ if and only if $b_1 \neq b_2$ or $w_1' \neq w_2'$.

Remark 3.1.2. Definition 3.1.8 is given constructively to ensure that if two finite words are unequal, then an explicit bit can act as a certificate for this inequality.

Definition 3.1.3. A *handle* is given by a **key**, a triple $(\mathrm{UID}, \mathrm{RID}, \mathrm{HID})$, where UID is a binary word called a **user ID**, RID is a binary word called the **revision ID**, and HID is a binary word called the **handle ID**.

[TODO[SMALL]: without getting into typing, enforce that equality has to be done on two words or two handles. Maybe lift to the latter to make sense?]

Definition 3.1.4. Consider two handles $h_1 \equiv (\mathrm{UID}_1, \mathrm{RID}_1, \mathrm{HID}_1), h_2 \equiv (\mathrm{UID}_2, \mathrm{RID}_2, \mathrm{HID}_2)$.

- h_1 and h_2 are equal, written $h_1 = h_2$, if and only if $\mathrm{UID}_1 = \mathrm{UID}_2$, $\mathrm{RID}_1 = \mathrm{RID}_2$, and $\mathrm{HID}_1 = \mathrm{HID}_2$.
- h_1 is not equal to h_2, written $h_1 \neq h_2$, if and only if at least one of the following hold: $\mathrm{UID}_1 \neq \mathrm{UID}_2$, $\mathrm{RID}_1 \neq \mathrm{RID}_2$, or $\mathrm{HID}_1 \neq \mathrm{HID}_2$.

Definition 3.1.5. A **unit** is defined recursively as a finite combination of the following and nothing else:

- A handle, see Definition 3.1.9.
- A representation $a \text{ — } c \to b$ of units a, b, c, where a is the **sign**, c is the **context**, and b is the **referent**.
- A block, which is defined as one of the following:
  - $\{\}$
  - Given a block g and unit u, $\{g, u\}$, $\{@g, u\}$, and $\{@g, \sim u\}$ are blocks, where $@g$ is a new blocks called the **expansion** of g and $\sim u$ is called the **exclusion** u.

Remark 3.1.6. In contrast to the requirement to the beginning of Li and Vitányi (see Section 2), the enumeration need *not* be surjective but only *locally* so. Abstracting away from the implicit meaning, units act as partial computable functions, but the latter is strictly *less* expressive by removing user provided meaning.

[TODO[SMALL]: maybe define alternation notation later or recursively?]

For notation, we will set $a \text{ — } c \to b \mid d$ to mean $\{a \text{ — } c \to b, a \text{ — } c \to d\}$ and $q \in c$ to mean $q \text{ — } c \to q$. Notice that many-to-many relationships are allowed.

[TODO[SMALL]: provide labels/links.]

[TODO[SMALL]: Maybe reduce the number of meta-variables used for clarity?]

[TODO[SMALL]: Clarify role of global context!]

DRAFT

Definition 3.1.7. The following rules apply, stated over meta-variables a, b, c, d, g, p, q:

- **R1. Internal Transitivity:** $a \text{ — } c \to b$ and $b \text{ — } c \to d$ imply $a \text{ — } c \to d$.
- **R2. Contextual Lifting:** $a \text{ — } c \to b$ and $p \xrightarrow{b} q$ imply $p \text{ — } a \to q \in c$.
- **R3. Empty:** $\{@g, \{\}\} \longleftrightarrow g$.
- **R4. Membership:** $\{@g, a\} \longleftrightarrow g$ if and only if $a \text{ — } g \to a$.
- **R5. Singleton:** $\{a\} \longleftrightarrow a$.
- **R6. Expansion:** if $a \text{ — } g \to b$, then $\{@g, p\} \leftrightarrow \{\{\{@g, a\}, b\}, p\}$.
- **R7. Exclusion:** if $g \longleftrightarrow \{@p, a\}$, then $\{\{@g, \sim a\}, b\} \longleftrightarrow \{@p, b\}$.
- **R8. Associativity:** $\{a, \{b, c\}\} \longleftrightarrow \{\{a, b\}, c\}$.
- **R9. Commutativity:** $\{\{@g, a\}, b\} \longleftrightarrow \{\{@g, b\}, a\}$.
- **R10. Null:** $\{a \text{ — } \{\} \to b\} \longleftrightarrow \{\}$.

We review the utility of each rule. Note that rules *between* contexts is entirely flexible and user defined. Moreover, only ? are needed for Turing completeness. We will show, however, that the remaining axioms are optimal when organizing information, see Section 6.

- **R1** and **R2** were discussed in Section 2.2.
- **R3** define the behavior of the empty unit $\{\}$, similar to the empty set.
- **R4** provides a way to represent $a \in g$ through the representation $a \text{ — } g \to a$. Because of this, $a \text{ — } g \to a$ may be a non-trivial path, so it is *not* required in every unit.
- **R5** reduces extraneous blocks. Note that this is *not* the same thing as the Quine atom, which states $\{a\} = a$ in a set theoretic context [Qui37]. We do *not* mean $a \in a$, but instead, $\{a\}$ is a *wrapper* around a. Notationally, we use extra blocks to identify *groups* of representations, such as $\{a \text{ — } b \to c, b \text{ — } c \to d\}$.
- **R6** and **R7** define how imports in the language work. An **import** is the process of joining the contents of one unit into another. **R6** says how imports work. **R7** provides a mechanism to *exclude* specified contents in g, which can themselves be arrows.
- **R-R** ensure that information can be repeated and is positionally invariant.

[TODO[SMALL]: note importance of using axioms to define essentially bound/free variables! Not as easy with just assuming sets as they are; easier to express the tree structure **first**.]

[TODO[SMALL]: clarify meta-variables! Maybe make these fixed **at** the start of this section? And probably make language clear?]

[TODO[MEDIUM]: double check all parts of proof!]

Theorem 3.1.8. *Any partial computable function is definable by a unit.*

Proof. We prove that we can embed any term in the SK-combinator calculus. This is an equational theory that is well known to be Turing complete [Cur30]. We provide on way to express this calculus in Welkin and prove it is correct.

To keep this proof self-contained, we define the full calculus:

- A **term** is defined recursively as either K or S, and if M, N are terms, so is (M) their **application** MN.
- Two terms M, N are **equal** if they are both K and S, or can be expressed as $M_1 M_2$ and $N_1 N_2$, where $M_1 = N_1$ and $M_2 = N_2$, respectively.
- For all terms M, N, $KMN = M$.

DRAFT

- For all terms M, N, P, $SMNP = MP(NP)$.

Now we can map each part of this into Welkin:

-
-

We now need to show the axioms for K and S hold:

- $KMN \longrightarrow M$:
- $SMNP \longrightarrow MP(NP)$:

□

3.2. PRA.

Definition 3.2.9. The relation u_1 is **contained in** u_2, denoted $u_1 < u_2$, is defined recursively:

- **Base case:** for each unit u, $u\neg < \{\}$.
- **Inductive step:** given units u_1 and $u_2 \equiv \{@g_1, a_2\}$, $u_1 < u_2$ if and only if either $u_1 < @g_1$ or for some $b \in g$, $u_1 < \{\{@g_1, \sim b\}, a\}$.

For universality, we need an important base construction that is definable in the theory: the ability to recurse through all IDs. From there, we can easily enumerate through all *potential* handles.

$$\text{bit} \longrightarrow 0 \mid 1$$
$$\text{word} \longleftrightarrow \{\text{head} \longrightarrow \{\text{bit} \mid \text{empty}\}, \text{next} \longrightarrow \text{word}\}$$

FIGURE 1. Generator for words in Welkin.

From there, we can define handle IDs through triples, see Figure 2.

$$\text{handle} \longleftrightarrow \{\text{UID} \longrightarrow \text{word}, \text{RID} \longrightarrow \text{word}, \text{HID} \longrightarrow \text{word}\}$$

FIGURE 2. Generator for handle keys in Welkin.

[TODO[MEDIUM]: show that R **correctly** distinguishes between all units!]

Definition 3.2.10. The **unit recursor** R is defined by $R \longleftrightarrow \{R_{\text{base}}, \text{Rules}\}$, where R_{base} is defined recursively, over meta-variables h, a, b, c:

- Includes Figure 1 and Figure 2.
- $\{\} \text{ — } R \rightarrow R$.
- For each handle h, $h \text{ — } R \rightarrow R$.
- For each arrow $a \text{ — } b \rightarrow c$, if $a, b, c \in R$, then $\{a \text{ — } b \rightarrow c\} \text{ — } R \rightarrow R$.
- **Monotonic:** for each a, b with $a \text{ — } R \rightarrow R$ and $b \text{ — } R \rightarrow R$, the following hold:
  - $a \text{ — } R \rightarrow \{a, b\}$.
  - $a \text{ — } R \rightarrow \{@a, b\}$.
  - $\{@a, \sim b\} \text{ — } R \rightarrow a$.

[TODO: make this discussion complete!] One interpretation of Definition 3.2.14 is defining Definition 3.2.13 *within* a unit, so R could be written as $<$ in the language. Moreover, R acts as the *least super bound* of all units.

Now we can prove the Turing definability of Welkin.

DRAFT

[TODO[MEDIUM]: Provide tabular proofs for these! Want to be very precise! But need to recognize when we are doing substitutions for meta-variables.]

The following theorems are two parts of the same **Recursion theorem** for Welkin.

Theorem 3.2.11. ***(Correctness)*** *For every unit u, $u \xrightarrow{R} R$, and if $u < u'$, then $u \longrightarrow u'$ and not $u' \longrightarrow u$ in R.*

Proof. We proceed on induction by units u:

- **Base case:** this is immediate, as $\{\}$ and all handles are included.
- **Inductive step:** there are four main cases:
  - $\{a \text{ — } b \to c\} \text{ — } R \to R$: this is immediate in context R.
  - $\{g, e\} \text{ — } R \to R$: suppose $g \longrightarrow R$ and $e \longrightarrow R$ in R. FINISH.
  - $\{@g, e\} \text{ — } R \to R$: suppose $g \longrightarrow R$ and $e \longrightarrow R$ in R. FINISH. use lifting?
  - $\{@g, \sim e\} \text{ — } R \to R$: assume $g \text{ — } R \to R$. Then, because $\{@g, e\} \text{ — } R \to g$, transitivity implies $\{@g, \sim e\} \text{ — } R \to R$.

□

[TODO[MEDIUM]: complete! And is this in the global context? Is this extensional equality or so?]

Theorem 3.2.12. ***(Uniqueness)*** *Let K that contains exactly $u \xrightarrow{K} u$ for each unit u. Then $K \leftarrow R \to R$.*

Proof. Assume for all units u, $u \xrightarrow{K} K$. Then $R \xrightarrow{R} R$. Thus, for any $p \text{ — } K \to q$ in K, $\{C \text{ — } K \to K, p \text{ — } K \to q\} \text{ — } K \to (p \text{ — } C \to q \in K)$

□

An important consequence of the recursion theorem is a basic form of **reflection**.

[TODO: make this precise! What does it mean for T to "contain the rules of Welkin"?]

Corollary 3.2.13. *Let T be any unit that extends the rules of Welkin. Then $\{T \longrightarrow R\} \xrightarrow{R} R$*

Proof. We proceed by induction, fixing the base to be R.

- **Base Case:** suppose T is a unit exactly with the rules $u \text{ — } T \to T$ for every unit u and the rules in Definition 3.1.12. Then by Theorem 3.2.3, $T \longleftrightarrow R$, completing the base case.
- **Inductive step:** suppose $T = \{T', e\}$ for a units T', e where $T' \longrightarrow R$. Now, by monotonicitiy in R, $T \longrightarrow T'$, hence by transitivity, $T \longrightarrow R$.

□

3.3. Expressing PA and Feferman Reflection.

The goal of this section is to completely define the following: *a proof of anything expressible as a RE set*. We will then generalize this to more through handles and mark soundness. Note that, because of Rice's Theorem, this method is inherently incomplete with computable methods. This theory does, however, establish a ceiling. As mentioned, we will postpone optimizations until Section 6.

DRAFT

3.4. **Coherency and Information.**

Definition 3.4.14.

Definition 3.4.15.

4. Syntax

We keep this section self-contained with explicit alphabets and recursive definitions. For consistency with Welkin, we write syntax using `type-writer font`. Notationally, we write $a_0, ..., a_n$ for a finite list of items, and use $a ::= a_1 \mid ... \mid a_n$ to denote a definition of a in terms of $a_1, ..., a_n$. For verification purposes, we will incorporate fixed bounds and completely unambiguous notation into Section 3.

4.1. **Words.**

Welkin's main encoding uses binary words, but add notation for decimal and hexadecimal.

```
bit ::= 0 | 1
digit ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
nibble ::= A | B | C | D | E | F
```

Listing 1. Binary, decimal, and hexadecimal digits.

A word is a sequence of digits, see Listing 2. We leave concatenation `.` as an undefined notion. We set concatenation to be right-associative, i.e., `(w.w').w'' = w.(w'.w'')`, and abbreviate `w.w'` as `w w'`. For conversions, see Definition 5.1.20.

```
word ::= binary | decimal | hex
binary ::= bit | binary.bit
decimal ::= digit | decimal.digit
hex ::= nibble | hex.nibble
```

Listing 2. Definition of words.

Equality is defined recursively, shown in Listing 3. Note that leading zeros are allowed.

[TODO[MEDIUM]: determine how to make Listing 3 work well in text **and** Welkin.]

```
0.word = word
0b0.word = word
0x0.word = word
```

Listing 3. Definition of word equality.

4.2. **Terminals.**

[TODO[SHORT]: determine whether to put these details in appendix. Maybe given the *overarching* grammar, at a high level?] Welkin uses ASCII as its base encoding. The term ASCII is slightly ambiguous, as there are subtly distinct variants, so we formally define US-ASCII as a standard version.[6]

[6]Note that this table *itself* is a representation, which represents glyphs with binary words. The use of these kinds of representations occur frequently in Welkin, see Section 3.

DRAFT

Definition 4.2.0. US-ASCII consists of 256 symbols, listed in Table 3.

To represent general encodings, there is a binary format supported for strings, see Listing 5.

Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph	Dec.	Hex.	Glyph
0	00	NUL	32	20	Space	64	40	@	96	60	`
1	01	SOH	33	21	!	65	41	A	97	61	a
2	02	STX	34	22	"	66	42	B	98	62	b
3	03	ETX	35	23	#	67	43	C	99	63	c
4	04	EOT	36	24	$	68	44	D	100	64	d
5	05	ENQ	37	25	%	69	45	E	101	65	e
6	06	ACK	38	26	&	70	46	F	102	66	f
7	07	BEL	39	27	'	71	47	G	103	67	g
8	08	BS	40	28	(	72	48	H	104	68	h
9	09	HT	41	29	)	73	49	I	105	69	i
10	0A	LF	42	2A	*	74	4A	J	106	6A	j
11	0B	VT	43	2B	+	75	4B	K	107	6B	k
12	0C	FF	44	2C	,	76	4C	L	108	6C	l
13	0D	CR	45	2D	-	77	4D	M	109	6D	m
14	0E	SO	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	|
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL

TABLE 3. US-ASCII codes and glyphs.

We denote specific characters through quotes, escaping if necessary. There are several important character classes in Listing 4, denoted through double quotes.

```
PRINTABLE  ::= [0x20-0x7E]
WHITESPACE ::= [0x09, 0x0A, 0x0D, 0x20]
DELIMITER ::= [0x7B, 0x7D, 0x2C, 0x2D, 0x2A, 0x3C, 0x3E, 0x22, 0x27,
0x5C, 0x7D]
```

LISTING 4. Important character classes.

DRAFT

Strings allow escaped single or double quotes, see Listing 5. IDs are special cases of strings that do not require quotes but forbid whitespace and certain characters, see Listing 6.

```
STRING ::= SQ_STRING | DQ_STRING
SQ_STRING ::= "'" (SQ_CHAR | ESCAPE_SQ )* "'"
DQ_STRING ::= "'" (DQ_CHAR | ESCAPE_DQ )* "'"

SQ_CHAR ::= PRINTABLE \ {'}
DQ_CHAR ::= PRINTABLE \ {"}
ESCAPE_SQ ::= "\'" | "\\"
ESCAPE_DQ ::= "\"" | "\\"
```

LISTING 5. Strings.

```
IMPORT ::= "@" ID
ID :: ID_CHAR+
ID_CHAR ::= PRINTABLE / (DELIMITERS + WHITESPACE + "#" + "@" + "'" +
"\"")
```

LISTING 6. IDs.

[TODO(MEDIUM): find good way to implement directly with Welkin or discuss.]

4.3. EBNF Notation and Parse Trees.

We define our variant of EBNF below:

Definition 4.3.1. An **EBNF** grammar consists of **productions**, which are pairs of the form `r ::= a_1 ... a_n`. On the right-hand side, juxtaposition means concatenation.

- Uppercase names require *no* whitespace between them. Otherwise, whitespace is allowed.
- `a ::= a_1 | ... | a_n` is short-hand for `{a ::= a_i | 1 <= i <= n}`.
- `(a_1)*` means zero or more instances of a_1.
- `(a_1)+` means one or more instances of a_1.
- `(a_1)?` means zero or one instance of a_1.

4.4. The Welkin Grammar.

Welkin's grammar is displayed in Listing 7, inspired by a minimal, C-style syntax. Note that when concatenating two terminals, denoted in uppercase, no whitespace between them is allowed, but in any other case, any amount of whitespace is allowed but ignored.

DRAFT

```
start        ::= terms
terms        ::= term ("," term)* ","? | EPS
term         ::= arc | graph | tuple | path
arc          ::= (term ("-" | "<-") term ("-" | "->"))+ term
graph        ::= path? "{" terms "}"
tuple        ::= path? "(" terms ")"

path         ::= MODIFIER? path_segment* unit
path_segment ::= unit | ".*" | "."+
unit         ::= ID | STRING


MODIFIER ::= "#" |  "@" | "~@" | "~"
ID         ::= ID_CHAR+
ID_CHAR    ::= PRINTABLE \ (DELIMITERS | WHITESPACE | "#" | "@" | "~" |
"'" | '"')
DELIMITERS ::= "," | "." | "-" | "<" | ">" | "*" | "(" | ")" | "{" | "}"
STRING     ::= SQ_STRING | DQ_STRING
SQ_STRING  ::= "'" (SQ_CHAR | ESCAPE_SQ )* "'"
DQ_STRING  ::= '"' (DQ_CHAR | ESCAPE_DQ )* '"'
SQ_CHAR    ::= PRINTABLE \ {"'"}
DQ_CHAR    ::= PRINTABLE \ {'"'}
ESCAPE_SQ  ::= "\'" | "\\"
ESCAPE_DQ  ::= '\"' | "\\"
PRINTABLE  ::= [0x20-0x7E]
WHITESPACE ::= [0x09, 0x0A, 0x0D, 0x20]
DELIMITER ::= [0x7B, 0x7D, 0x2C, 0x2D, 0x2A, 0x3C, 0x3E, 0x22, 0x27,
0x5C, 0x7D]
EPS        ::= ""
```

LISTING 7. The grammar for Welkin. The terminals `id` and `string` are defined in Listing 2 and Listing 5, respectively

[TODO(HIGH): determine if this is good as is or needs to be put into bootstrap!]

4.5. **Proof of Unambiguity.**

We now prove that the Welkin language is unambiguous by showing it is LL(1), a rich class of grammars that can be efficiently parsed. For more details, please consult [Aho+06].

Moreover, we define the top of a word in Listing 8.

```
top(word) ::= nil => nil | bit.word => bit
```

LISTING 8. Definition of the top of a word.

Definition 4.5.2. ([RS70]). A grammar is LL(1) iff the following holds: for any terminal w_1 and nonterminal A, there is at most one rule r such that for some w_2, w_3 appearing at the top of A such that,

- $S \Rightarrow \mathrm{top}(w_1)Aw_3$
- $A \Rightarrow w_2(p)$
- $\mathrm{top}(w_2w_3) = w$

Theorem 4.5.3. *There exists some* LL(1) *grammar that accepts the same strings as the Welkin grammar Listing 7. Hence, Welkin's syntax is unambiguous, i.e., every string accepted by the language has exactly one derivation.*

DRAFT

Proof. We use transformations in Table 4 that preserve the language of the original grammar, resulting in Listing 9. For the refactor step by step, see Table 5. We can readily verify that there are no shared prefixes for a single production, see Table 6. Because there are no conflicts, the transformed grammar is LL(1), and hence, the grammar is unambiguous.

Rule ID	Name	Description
T0	**Group Flattening**	Converts Kleene stars `A*` and regex-like lists into right-recursive forms `A' ::= A A' | EPS`.
T1	**Left Refactoring**	Transforms overlapping prefixes `A ::= B C | B D` into `A ::= B (C | D)` to eliminate FIRST set collisions.
T2	**Lexical State Expansion**	Expands complex sequence operators (`+`, `*`) into strict right-recursive terminal rules, ensuring contiguous consumption without whitespace interruptions.
T3	**Left-Recursion Removal**	Eliminates immediate left-recursion `A ::= A B | C` by rewriting as `A ::= C A'` and `A' ::= B A' | EPS` to prevent infinite loops.

TABLE 4. Well known transformations on grammars that preserve string acceptance.

DRAFT

```
start      ::= terms
terms      ::= term terms_tail | EPS
terms_tail ::= "," terms | EPS
term       ::= node chain

chain      ::= left_link node right_link
               node chain | EPS

left_link  ::= "-" | "<-"
right_link ::= "-" | "->"

path     ::= path_segment* unit
path_segment ::= MODIFIER? (UNIT | ".*" | "."+)

node       ::= PATH opt_block | block
opt_block  ::= block | EPS
block      ::= "{" terms "}"
             | "(" terms ")"

PATH       ::= MODIFIER PATH_BODY
             | PATH_BODY
PATH_BODY  ::= "." PATH_DOTS
             | UNIT PATH_TAIL
PATH_DOTS  ::= "*" PATH_BODY
             | "." PATH_DOTS
             | UNIT PATH_TAIL
PATH_TAIL  ::= PATH_BODY | EPS

UNIT     ::= IMPORT | ID | STRING
MODIFIER   ::= "#" | "@" | "~@" | "~"
ID         ::= ID_CHAR+
ID_CHAR    ::= PRINTABLE \ (DELIMITERS | WHITESPACE | "#" | "@" | "~" |
"'" | '"')
DELIMITERS ::= "," | "." | "-" | "<" | ">" | "*" | "(" | ")" | "{" | "}"
STRING     ::= SQ_STRING | DQ_STRING
SQ_STRING  ::= "'" (SQ_CHAR | ESCAPE_SQ )* "'"
DQ_STRING  ::= '"' (DQ_CHAR | ESCAPE_DQ )* '"'
SQ_CHAR    ::= PRINTABLE \ {"'"}
DQ_CHAR    ::= PRINTABLE \ {'"'}
ESCAPE_SQ  ::= "\'" | "\\"
ESCAPE_DQ  ::= '\"' | "\\"
EPS        ::= ""
```

LISTING 9. Transformed LL(1) grammar for Welkin, with all terminals defined.

DRAFT

Original	Transform	LL(1)

`start ::= terms,`
`terms ::= term ("," term)* "","? | EPS`
	Transform 1	`start ::= terms`
		`terms ::= term terms_tail | EPS`
		`terms_tail ::= "," terms | EPS`

`term ::= arc | graph | group | path`
`arc ::= (term ("-" | "<-") term ("-" | "->"))+ term`
	Transform 4	`/* Extracted 'node' to fix recursion.`
		`Arcs are strict left/right link pairs */`
		`term ::= node chain`
		
		`chain ::= left_link node right_link node chain | EPS`
		
		`left_link ::= "-" | "<-"`
		`right_link ::= "-" | "->"`

`graph ::= path? "{" terms "}"`
`tuple ::= path? "(" terms ")"`
`path ::= modifier? path_segment* unit`
	Transform 2, Transform 3	`/* Left-factor path & blocks. */`
		`node ::= PATH opt_block | block`
		`opt_block ::= block | EPS`
		`block ::= "{" terms "}" | "(" terms ")"`
		
		`/* Expand path +, * contiguously */`
		`PATH ::= MODIFIER PATH_BODY | PATH_BODY`
		`PATH_BODY ::= "." PATH_DOTS | UNIT PATH_TAIL`
		`PATH_DOTS ::= "*" PATH_BODY | "." PATH_DOTS | UNIT PATH_TAIL`
		`PATH_TAIL ::= PATH_BODY | EPS`

TABLE 5. Refactor of grammar Listing 7 into Listing 9. Entries with - mean that no changes are needed.

DRAFT

Non-Terminal	Lookahead (a)	Production Chosen
start	"#" | "@" | "~@" | "~" | "." | ID | STRING | "{" | "(" | EOF	terms
terms	"#" | "@" | "~@" | "~" | "." | ID | STRING | "{" | "("	term terms_tail
	EOF | "}" | ")"	EPS
terms_tail	","	"," terms
	EOF | "}" | ")"	EPS
term	"#" | "@" | "~@" | "~" | "." | ID | STRING | "{" | "("	node chain
node	"#" | "@" | "~@" | "~" | "." | ID | STRING	PATH opt_block
	"{" | "("	block
opt_block	"{" | "("	block
	EOF | "}" | ")" | "," | "-" | "<-" | "->"	EPS
block	"{"	"{" terms "}"
	"("	"(" terms ")"
chain	"-" | "<-"	left_link node right_link node chain
	EOF | "}" | ")" | ","	EPS
left_link	"-"	"-"
	"<-"	"<-"
right_link	"-"	"-"
	"->"	"->"
PATH	"#" | "@" | "~@" | "&"	MODIFIER PATH_BODY
	"." | ID | STRING	PATH_BODY
PATH_BODY	"."	"." PATH_DOTS
	ID | STRING	UNIT PATH_TAIL
PATH_DOTS	"*"	"*" PATH_BODY
	"."	"." PATH_DOTS
	ID | STRING	UNIT PATH_TAIL
PATH_TAIL	"." | ID | STRING	PATH_BODY
	EOF | "}" | ")" | "," | "-" | "<-" | "->" | "{" | "("	EPS
MODIFIER	"#"	"#"
	"@"	"@"
	"~@"	"~@"
	"&"	"&"
UNIT	ID	ID
	STRING	STRING

TABLE 6. LL(1) Table for Listing 9

□

5. SEMANTICS

This section describes several phases to transform parse trees into more refined forms called **Internal Representations (IR)**. These phases are:

- Abstract Syntax Trees (ASTs): simplifies the parse tree and removes punctuation.

DRAFT

- Lexicographic Ordering: Lexicographically orders graphs by names and anonymous graph

content.

- Unique IDs: Assigns IDs to all names and resolves absolute and relative paths.
- Merging: merges units and defines the final scopes.

How ASTs are processed and validated. We postpone information organization to Section 6.

5.1. **Abstract Syntax Tree (ASTs).**

Given the rationale, we explain how the Abstract Syntax Tree (AST) is processed for the syntax. The AST provides an intermediate step before the final data structure.

Definition 5.1.0. The **Abstract Syntax Tree** is recursively defined from the parse tree of Listing 7 as follows:

- **Terms:** Converted into a list, which is empty if `EPS` is matched.
- **Term:** either a Root, Arc, Graph, Group, or Path, with two additional fields:
  - **Position:** a pair $(\mathrm{Line}, \mathrm{Column})$, where Line is the first

number of newline ("n") characters occurring before the term and Column is the position of this term on the line. Both of these are stored as bytes.

- **Root:** simply stores the corresponding unit.
- **Arc:** This is converted into a list. The first item is $(s_0, c_0 r_0)$, the first triple that occurs in the chain. Then, the remaining triples are added to the list.
  - Left arrows are added as (r_0, c_0, r_0). Edges and double arrows are added as both a left and right arrow.
- **Graph:** The terms are collected into two parts: a list of parts and a list of arcs. Each graph has a name; when no name is provided, it is `""`.
- **Tuple:** The terms are organized recursively, with the base case starting

at `item` and the recursive step at the label `next`. Note that tuples have **closed** definitions and will create copies when accessed or used in an arc.

- **Path:**
  - The number of dots is counted for the relative paths.
  - Star imports are denoted by a special node All.
  - A path is converted into a list of its contents,

  which are pairs containing the relative path number and either Unit or All.
  - The `unit` is added at the end.
- **ID:** converted into strings.
- **String:** Wraps around the contents.
- **Number:** converts decimal and hexadecimal into binary, recursively over words according to Table 7.

The terms in the top-level are put into a Graph node containing a unique, user given ID.

[TODO[SHORT]: determine nice way to merge this with Listing 3!]

DRAFT

Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Dec	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Bin	0	1	10	11	100	101	110	111	1000	1001	1010	1011	1100	1101	1110	1111

TABLE 7. Conversions of digits between different bases.

Definition 5.1.1. An Abstract Syntax Tree is **valid** if the following holds:

- A Root term must exist. Moreover, there must not be conflicting Root term names.
- Relative imports does not exceed the number of available parents.

Remark 5.1.2. An earlier revision of this thesis forbid repetitions of arcs and units. However, this restriction was removed to provide greater flexibility. This will be tracked, see ?.

5.2. **Unified IDs.**

This phase first lexicographically orders the graph by its labels. Anonymous graphs are lexicographically ordered by contents, with arcs treated as triples and lexicographically ordered accordingly. Then, IDs are assigned. The lexicographic ordering ensures the ID is *exactly* the same for two strings that are positionally different. This shows that Welkin is positionally invariant.

5.3. **Unification.**

This phase merges the units into the final data structure.

[TODO[SHORT]: add link to quote!]

Remark 5.3.3. In contrast to the requirement to the beginning of Li and Vitány (see Section 2), the enumeration need *not* be surjective but only *locally* so. Abstracting away from the implicit meaning, units act as partial computable functions, but the latter is strictly *less* expressive by removing user provided meaning.

[TODO: ensure this definition is general enough! We will need to tackle the third rule, having unspecified parameters, more in depth. Does this mean an implementation defined feature? Or does it generalize it?] However, this is only one component: we also must prove we can represent *any* truth management system. This is made possible through contexts. We define a **truth management system** generally as a partial computable function augmented with parameters that denote the truth of base statements or **axioms**. These are intentionally left undefined, in the same vein as **R3**. In fact, by **R3** and Theorem 3.1.1, we obtain the following.

Corollary 5.3.4. *Any computable truth management system can be represented as a*

Note that it is essential to have contexts via **R2**, as shown by the following.

Theorem 5.3.5. *Representations with contexts cannot be expressed with those without.*

Proof. The largest class expressible with unconditional representations are context-free grammars, because… Thus, not all partial computable functions are included, completing the proof. □

DRAFT

5.4. **Queries and Information.**

We set $\left(u \xrightarrow{v} w\right)_c \in x \Leftrightarrow x(u) \xrightarrow{x(v)} x(w)$, where $x(s)$ is the local extension of s in x. We interpret $u \longrightarrow v$ as: the **sign** u represents **referent** v in **context** c. Through Theorem 3.1.1, we will present the following computational interpretation:

$$u \xrightarrow{v} w \text{ iff } \varphi_u(v) \text{ evaluates to } w,$$

where φ_u is the partial computable function given by the ID of u. Note that this is *only* logical equivalence; the former is strictly *more* expressive, due to implicit bindings.

Definition 5.4.6. A unit u is **non-trivial** if it is non-empty and has a non-complete representation graph. A unit u is **coherent relative to a context** u' if $u + u'$, the union of these units, is non-trivial.

Remark 5.4.7. This definition is a natural generalization of consistency in first-order logic. We will frequently rely on this result throughout the thesis.

Definition 5.4.8. Let u, v be units. Then u **contains information** v if for some $s \in v$, $u[s] \neq s$.

Our notion of information helps with one key issue: the general undefinability of non-trivial classes of partial computable functions in formal system. This connects with the absence of a universal *single* formal system that can prove any claim about, e.g., Peano Arithmetic.

[TODO: clean up this example. Want to emphasize what is information here, so, e.g., we may say left and right nodes don't have information about each other, in general]

Example 5.4.9. Consider the recursive definition of a binary tree: either it is a null (leaf) node, or it contains two nodes, left and right. We can model this as follows:

- First, create units for each of the notions: `tree {null, left, right}`.

[TODO: add a condition that the left and right trees are distinct, to show this is possible!]

- Next, we write, `tree { nil --> .tree, left..tree, right..tree, {.left, .right} --> .tree}`. Notice that we refer to the *namespace* via a relative path, `.tree`, thereby enabling recursion.
- We can test this out in Welkin with: `my_tree {.tree.left --> {nil --> .tree}, .tree.right {nil --> .tree} }`. This is then coherent with the previous definition.

Are are two important ideas in this example. First, an abstraction can be defined prior to a concrete model. The other way is possible as well, showing how developing representations are flexible in Welkin. Second, the derivations of trees can now be formulated. So we can define descendants and ancestors, and test against the coherency of the tree.

A key technique in managing information and truth through contexts is through the following theorem. FIXME: this is currently a stub! Need to create the **correct** condition. Use this as a starting point:

Theorem 5.4.10. *A unit u contains information about v iff $u + v$ is coherent.*

DRAFT

[TODO: Develop the notion of a query and its relation to information. Ultimately, we want to define information based on how useful it is for querying the database. We want to define a query to be anything we can inquire *about* a database that we we can (partially) computably represent. Information should then follow quickly from there as a *partial* answer. Having *enough direct* information means being able to *fully* solve the query. Moreover, the goal will be to use that this notion of enough is efficient, so checking for this should be efficient, say $O(n)$ or $O(n^2)$. This will likely be based on rewrite rules, in combination with axiom **R3**.]

[TODO: a core part of queries is **indirect** information, or information that may not be directly visible by immediately applying rewrite rules. This relates to my earlier attempts on a universal progress theorem. What I want is to know if query is "well-posed" or *can* be solved by a computable representation. This well-posedness needs to be *defined* in the truth system itself, and part of this may be undecidable. However, most of this should be converted to *finitistic properties of comptuable functions*. This will be my new version of "universal progress", and I may provide an example where introducing *bridging* representations may be effective, such as through mathematics and music (which is *not* necesarially a homomorphism between proof systems.)]

6. Information Organization

The presentation of Welkin's universal expressivity, stated as Theorem 3.1.1 fixed with one particular representation. Following the analogue of units to practical computable functions, we define **Universal Representation Systems (URS)** as the analogues of Universal Turing Machines, see Definition 6.1.25.

A major problem for scalability is *choosing* a URS. Possibly the use of multiple URSs for different use cases is more optimal, in some sense? The key operation in an information base is *querying*, so this must be as efficient as possible. . This As established in Section 5, bounded queries can be answered in $O(?)$ time. The problem then becomes about optimizing the number of steps. While this is query dependent, and depends on the database, we prove that any of these criterion can be converted to one about *size*. Our proof generalizes Blum's axioms [Blu67] and Kolomogorov complexity [LV19]. While finding the absolute smallest size of a unit that will best optimize a query is impossible, we *can* optimize the database with the available information. Our localized algorithm provides a nice architecture to solve problems: combining bounded queries in the database to confirm the presence of an answer, combined with unbounded searches by some search procedure or heuristics. Note that the search procedure may or may not be computable; what is important is that bounded queries are always efficient. We also provide proof certificates.

6.1. **Universal Systems.**

Note that there are multiple ways to prove Theorem 3.1.1, infinitely in fact. This motivates the following definition.

Definition 6.1.0. A universal representation system (URS) is a unit that can represent any representation.

DRAFT

Theorem 6.1.1. *A unit is a universal representation system if and only if it can represent any partial computable function. Moreover, any universal representation system can represent any universal representation system. In particular, representing itself is called* ***reflection****.*

[TODO: dissue axiomatic systems! Want to emphasize the relevant **process** (per context) is important! That is, the journey to discover new things. ONLY FI the specification is complete in some way (or "finalized"), it is then that axiomatic systems **can** help. Expand this discussion into a paragraph or two.]

The term *universal* is specifically for expressing *representations* symbolically. The free parameter still needs to be included and is an additional feature on top of partial comptuable functions. However, the *management* of these symbols is done entirely with partial computable functions.

The next section discusses the issue of *managing* the infinitely many choices for URSs.

6.2. Localized Size Compression.

Instead of making proofs most efficient as is, we want to support finding optimal representations. But we want to do this from an efficiently queryable system, which *is* the most optimal.

7. Conclusion

[TODO: reformat this to write: foundations, syntax, semantics.] This thesis introduced Welkin, a universal, formalized information language. The syntax (Section 4) was defined rigorously with a small EBNF, shown to be accepted by an LL(1) grammar, showing that parsing is unambiguous. The semantics (Section 5) were provided with several passes to convert parse trees into units, which contain both a hierarchical and relational structure for scoping and direct representations, respectively. Units have key properties that enable them to express any partial computable function Theorem 3.1.1, in conjungtion with expressing any truth management system, demonstrates **universality** of the system. This is practically demonstrated by showing that all the major paradigms in Information Management and Knowledge Management are expressible within Welkin. Moreover, it was shown that there is a way to best organize the language given available information Section 6, showing **scalability**. Finally, the bootstrap in Section 3 self-hosts the language within a bounded 64 variant, whose complete Unambiguity (as well as the grammar's prior) establishes **standardization**. Revisions further enhance this by

The remaining sections show several areas for future work. This list is not exhaustive and, by the previous arguments, and can be applied to *any* subject with computable representations (essentially, any human subject).

7.1. Comparisons to Existing Literature.

In addition to domain specific proposals, there are approaches for general IM which still fail to resolve all issues. One prominent example is Burgin's General Theory of Information [Bur09] that comprehensively includes many separate areas for IM, including the complexity-based Algorithmic Information Theory, through a free parameter called an "infological system", which encompasses domain specific terminology and concepts. In contrast to other approaches, Burgin's generalized theory is flexible

DRAFT

and enables greater coverage of different kinds of information [Mik23]. Despite this coverage Burgin does not closely tie the free parameter with his formal analysis of Algorithmic Information Theory, making it unclear how to use this in a practical implementation. Burgin's Theory of Information, along with broad proposals, have severe shortcomings, highlighting major obstacles for IM.

7.2. **Programming Languages and Formal Verification.**

[TODO: add discussions on supporting interoperability with languages, providing more robust and unified implementations of core libraries, compilers, and operating systems.]

Taking the enable custom hardware implementations for checking and reduce the surface area for attacks on verifying certifications for many applications, including cryptography.

Moreover, the proposed architecture could use an LLM as an oracle.

7.3. **Mathematical and Scientific Knowledge.**

[TODO: discuss more applications of Welkin in depth.]

There are several possible projects to pursue in mathematics and scientific research. For mathematics, there are several existing rojects for storing mathematical information (see [CF09] for more details). Older proposals, including the QED Manifesto [KR16] and the Module system for Mathematical Theories (MMT), aimed to be more general and have seen limited success. More centralized systems, like `mathlib` in the Lean proof assistant [The20], have seen adoption but do not give equal coverage nor are interoperable with other systems. Welkin enables this interopreeable through gradual translations, and with Section 6, one can always determine if there is enough *direct* information to complete a translation. This will help facilitate reusability among major tools, and aid in formal verification (Section 7.2) well.

Along with mathematics, Welkin could provide more rigorous frameworks for the sciences, with are currently scattered with different proposals. One prominent proposal is the Findable Accessible Interoperable Reusable (FAIR) guidelines [Wil+16]. Instead of providing a concrete specification or implementation, FAIR provides best practices for storing scientific information. However, multiple papers have outlined problems with these overarching principles, including missing checks on data quality [Gui+25], missing expressiveness for ethics frameworks [Car+21], and severe ambiguities that affect implementations [Jac+20]. Welkin addresses these by using contexts strategically. Experiments can be compared using revisions, and disagreements between experts can be analyzed using separate contexts. These contexts can then *distinguish* between different theories, and scientists can select the unit with the best or most comprehensive evidence. Metrics for such evidence can be *representable* to a certain point, but at a minimum, they can be more effectively analyzed.

7.4. **Humanities.**

[TODO: significantly expand.]

Information Management in the humanities has few models, including an adaption of FAIR [Har+20] and discipline specific, linked databases in the PARNTHEOS project [Hed+19]. Welkin could assist by providing a space to help standardize this and localize different publication styles and literary theories.

Draft

References

[Ack89] Ackoff, R L, From Data to Wisdom, Journal of Applied Systems Analysis **16** (1989) 3–9.

[Aho+06] Aho, Alfred V., Lam, Monica S., Sethi, Ravi, and Ullman, Jeffrey D., *Compilers: Principles, Techniques, and Tools (2nd Edition)*, Addison Wesley, 2006.

[Ang18] Angles, Renzo, The Property Graph Database Model., in *AMW* (Olteanu, Dan and Poblete, Barbara (eds.), CEUR-WS.org.

[AGH05] Arnold, Ken, Gosling, James, and Holmes, David, *The Java Programming Language*, 4th ed., The Java Series, Addison-Wesley Professional, Upper Saddle River, NJ, 2005.

[Art24] Artemov, Sergei, Serial properties, selector proofs and the provability of consistency, Journal of Logic and Computation **35** no. 3 (2024) exae34.

[Atk23] Atkin, Albert, Peirce's Theory of Signs, Spring2023 ed., Metaphysics Research Lab, Stanford University, 2023.

[Bat00] Bateson, Gregory, *Steps to an ecology of mind*, University of Chicago Press ed., University of Chicago Press, Chicago, 2000.

[Blu67] Blum, Manuel, A Machine-Independent Theory of the Complexity of Recursive Functions, J. Acm **14** no. 2 (1967) 322–336.

[Buc91] Buckland, Michael K., Information as thing, Journal of the American Society for Information Science **42** no. 5 (1991) 351–360.

[Bur09] Burgin, Mark, *Theory of Information*, World Scientific, 2009.

[BF95] Buvač, Saša, and Fikes, Richard, A declarative formalization of knowledge translation, in *Proceedings of the Fourth International Conference on Information and Knowledge Management*, Association for Computing Machinery, Baltimore, Maryland, USA, pp. 340–347.

[CF09] Carette, Jacques, and Farmer, William M., A Review of Mathematical Knowledge Management, in *Intelligent Computer Mathematics* (Carette, Jacques, Dixon, Lucas, Coen, Claudio Sacerdoti, and Watt, Stephen M. (eds.), Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 233–246.

[Car+21] Carroll, Stephanie Russo, Herczog, Edit, Hudson, Maui, Russell, Keith, and Stall, Shelley, Operationalizing the CARE and FAIR Principles for Indigenous Data Futures, Scientific Data **8** no. 1 (2021) 108.

[Cur30] Curry, H. B., Grundlagen der Kombinatorischen Logik, American Journal of Mathematics **52** no. 3 (1930) 509–536.

[Dic25] Dictionary, Oxford English, welkin, n., Oxford University Press, 2025.

[Fef62] Feferman, Solomon, Transfinite Recursive Progressions of Axiomatic Theories, The Journal of Symbolic Logic **27** no. 3 (1962) 259–316.

[Fin85] Fine, Kit, *Reasoning with Arbitrary Objects*, Blackwell, New York, NY, USA, 1985.

[Flo10] Floridi, Luciano, *Information : a very short introduction*, Oxford University Press, Oxford ; New York, 2010.

[GWW07] Goczyla, Krzysztof, Waloszek, Wojciech, and Waloszek, Aleksander, Contextualization of a DL Knowledge Base., p. .

[Gui+25] Guillen-Aguinaga, Miriam, Aguinaga-Ontoso, Enrique, Guillen-Aguinaga, Laura, Guillen-Grima, Francisco, and Aguinaga-Ontoso, Ines, Data Quality in the Age of AI: A Review of Governance, Ethics, and the FAIR Principles, Data **10** no. 12 (2025).

[Har90] Harnad, Stevan, The Symbol Grounding Problem, Physica D **42** (1990) 335–346.

[Har07] Harrington, Brian, ASKNet: Automated Semantic Knowledge Network., in *AAAI*, AAAI Press, pp. 1862–1863.

[Har+20] Harrower, Natalie, Quinn, Mary, Tóth-Czifra, Erzsébet, and others, *Sustainable and FAIR Data Sharing in the Humanities: Recommendations of the ALLEA E-Humanities Working Group*, Berlin, 2020.

[HS13] Hazarika, Shyamanta, and Sinha, Smriti, Formal concept analysis: current trends and directions, Artificial Intelligence Review **44** (2013) 1–40.

[Hed+19] Hedges, Mark, Stuart, David, Tzedopoulos, George, Bassett, Sheena, Garnett, Vicky, Giacomi, Roberta, and Sanesi, Maurizio, *Digital Humanities Foresight: The future impact of digital methods, technologies and infrastructures*, GOEDOC, Dokumenten-und Publikationsserver der Georg-August-Universität Göttingen, 2019.

DRAFT

[Hit21] Hitzler, Pascal, A review of the semantic web field, Commun. ACM **64** no. 2 (2021) 76–83.

[Hit+09] Hitzler, Pascal, Krötzsch, Markus, Parsia, Bijan, Patel-Schneider, Peter F., and Rudolph, Sebastian, *OWL 2 Web Ontology Language Primer*, 2009.

[Jac+20] Jacobsen, Annika, Miranda Azevedo, Ricardo de, Juty, Nick, Batista, Dominique, Coles, Simon, Cornet, Ronald, Courtot, Mélanie, Crosas, Mercè, Dumontier, Michel, Evelo, Chris T., Goble, Carole, Guizzardi, Giancarlo, Hansen, Karsten Kryger, Hasnain, Ali, Hettne, Kristina, Heringa, Jaap, Hooft, Rob W.W., Imming, Melanie, Jeffery, Keith G., Kaliyaperumal, Rajaram, Kersloot, Martijn G., Kirkpatrick, Christine R., Kuhn, Tobias, Labastida, Ignasi, Magagna, Barbara, McQuilton, Peter, Meyers, Natalie, Montesanti, Annalisa, Reisen, Mirjam van, Rocca-Serra, Philippe, Pergl, Robert, Sansone, Susanna-Assunta, Silva Santos, Luiz Olavo Bonino da, Schneider, Juliane, Strawn, George, Thompson, Mark, Waagmeester, Andra, Weigel, Tobias, Wilkinson, Mark D., Willighagen, Egon L., Wittenburg, Peter, Roos, Marco, Mons, Barend, and Schultes, Erik, FAIR Principles: Interpretations and Implementation Considerations, Data Intelligence **2** nos. 1–2 (2020) 10–29.

[KBp] KBpedia, OpenCyc Resources.

[KR16] Kohlhase, Michael, and Rabe, Florian, QED Reloaded: Towards a Pluralistic Formal Library of Mathematical Knowledge, Journal of Formalized Reasoning **9** (2016) 201–234.

[Len95] Lenat, Douglas B, CYC: A large-scale investment in knowledge infrastructure, Communications of the ACM **38** no. 11 (1995) 33–38.

[Len+90] Lenat, Douglas B., Guha, R. V., Pittman, Karen, Pratt, Dexter, and Shepherd, Mary, Cyc: toward programs with common sense, Commun. ACM **33** no. 8 (1990) 30–49.

[LV19] Li, Ming, and Vitanyi, Paul, *An Introduction to Kolmogorov Complexity and Its Applications*, 4th ed., Springer Publishing Company, Incorporated, 2019.

[The20] The mathlib Community, The lean mathematical library, in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, Association for Computing Machinery, New Orleans, LA, USA, pp. 367–381.

[McC93] McCarthy, John, Notes on formalizing context, in *Proceedings of the 13th International Joint Conference on Artifical Intelligence - Volume 1*, Morgan Kaufmann Publishers Inc., Chambery, France, pp. 555–560.

[Mes+15] Mesgari, Mostafa, Okoli, Chitu, Mehdi, Mohamad, Nielsen, Finn Årup, and Lanamäki, Arto, ``The sum of all human knowledge'': A systematic review of scholarly research on the content of Wikipedia, Journal of the Association for Information Science and Technology **66** no. 2 (2015) 219–245.

[Mik23] Mikkilineni, Rao, Mark Burgin's Legacy: The General Theory of Information, the Digital Genome, and the Future of Machine Intelligence, Philosophies **8** no. 6 (2023).

[MDV26] Mora, Angel, Dominguez, Jesus, and Villalon, Ana, fcaR: Formal Concept Analysis, 2026.

[Qui37] Quine, W. V., New Foundations for Mathematical Logic, The American Mathematical Monthly **44** no. 2 (1937) 70–80.

[Rei17] Reisman, Miriam, EHRs: The Challenge of Making Electronic Data Usable and Interoperable, P & T : a peer-reviewed journal for formulary management **42** (2017) 572–575.

[RO10] Rendel, Tillmann, and Ostermann, Klaus, Invertible syntax descriptions: unifying parsing and pretty printing, in *Proceedings of the Third ACM Haskell Symposium on Haskell*, Association for Computing Machinery, Baltimore, Maryland, USA, pp. 1–12.

[RWE15] Robinson, Ian, Webber, Jim, and Eifrem, Emil, *Graph Databases*, 2nd ed., O'Reilly Media, Inc., 2015.

[RS70] Rosenkrantz, D.J., and Stearns, R.E., Properties of deterministic top-down grammars, Information and Control **17** no. 3 (1970) 226–256.

[SL17] Smith, Brent, and Linden, Greg, Two Decades of Recommender Systems at Amazon.com, IEEE Internet Computing **21** no. 3 (2017) 12–18.

[Str13] Stroustrup, Bjarne, *The C++ Programming Language*, 4th ed., Addison-Wesley Professional, Boston, MA, 2013.

[Tur36] Turing, Alan M., On Computable Numbers, with an Application to the Entscheidungsproblem, Proceedings of the London Mathematical Society **2** no. 42 (1936) 230–265.

[Wil+16] Wilkinson, Mark D., Dumontier, Michel, Aalbersberg, IJsbrand Jan, Appleton, Gabrielle, Axton, Myles, Baak, Arie, Blomberg, Niklas, Boiten, Jan-Willem, Silva Santos, Luiz Olavo Bonino da, Bourne, Philip E., Bouwman, Jildau, Brookes, Anthony J., Clark, Tim, Crosas, Mercè, Dillo, Ingrid, Dumon, Olivier, Edmunds, Scott C., Evelo, Chris T. A., Finkers, Richard, González-Beltrán, Alejandra N., Gray, Alasdair J. G., Groth, Paul, Goble, Carole A., Grethe, Jeffrey S., Heringa, Jaap, Hoen, Peter A. C. 't, Hooft, Rob W. W., Kuhn, Tobias, Kok, Ruben G., Kok, Joost N., Lusher, Scott J., Martone, Maryann E., Mons, Albert, Packer, Abel Laerte, Persson, Bengt, Rocca-Serra, Philippe, Roos, Marco, Schaik, Rene C. van, Sansone, Susanna-Assunta, Schultes, Erik Anthony, Sengstag, Thierry, Slater, Ted, Strawn, George O., Swertz, Morris A., Thompson, Mark, Lei, Johan van der, Mulligen, Erik M. van, Velterop, Jan, Waagmeester, Andra, Wittenburg, Peter, Wolstencroft, Katy, Zhao, Jun, and Mons, Barend, The FAIR Guiding Principles for scientific data management and stewardship, Scientific Data **3** (2016).

[Zhu+23] Zhu, Xixi, Liu, Bin, Zhu, Cheng, Ding, Zhaoyun, and Yao, Li, Approximate Reasoning for Large-Scale ABox in OWL DL Based on Neural-Symbolic Learning, Mathematics **11** no. 3 (2023).

Department of Mathematics, University of Colorado at Boulder, Boulder, CO
Email address: oscar-bender-stone@protonmail.com

DRAFT